

EMSI Casablanca
École Marocaine des Sciences de l'Ingénieur
Filière : Ingénierie Informatique et Réseaux

Conception, implémentation, comparaison et analyse critique de modèles de Deep Learning

pour données tabulaires, images et séquences

Module : Deep Learning
Projet de fin de module – Évaluation finale (travail individuel)

Réalisé par :
Essadik Mohamed Yassir

Encadré par :
Mme HIDILA Zineb

Résumé

Ce rapport présente la conception, l’implémentation sous **PyTorch** et l’analyse comparative de trois familles d’architectures de *deep learning*, chacune adaptée à une nature de données différente : un **perceptron multicouche (MLP)** pour la classification de données tabulaires (*Breast Cancer Wisconsin*), un **réseau de neurones convolutif (CNN)** pour la classification d’images (*CIFAR-10*), et des **modèles récurrents (RNN, LSTM, GRU)** ainsi qu’un **système Seq2Seq** encodeur–décodeur pour la modélisation du langage et la traduction automatique anglais→français. Pour chaque partie, nous articulons les fondements théoriques, une méthodologie expérimentale rigoureuse (séparation train/validation/test, prévention du *data leakage*, reproductibilité par graine aléatoire) et une analyse critique des résultats. Au-delà de la simple performance, nous étudions expérimentalement les *choix de conception* : stratégies d’initialisation, ablations architecturales (padding, stride, pooling, convolution 1×1), effet du *gradient clipping*, et stratégies de décodage (glouton vs *beam search*). La discussion transversale montre comment un même paradigme d’apprentissage supervisé se décline en *biais inductifs* différents selon la géométrie, la dépendance locale et la temporalité des données.

Mots-clés : Deep Learning, PyTorch, MLP, CNN, RNN, LSTM, GRU, Seq2Seq, biais inductif, BLEU, perplexité, interprétabilité.

Note de lecture. Les valeurs numériques en **rouge** sont des *ordres de grandeur attendus*. Elles doivent être remplacées par les résultats exacts obtenus après exécution des trois notebooks sur Google Colab (GPU). Voir l’annexe B pour les noms de fichiers de figures attendus.

Table des matières

Résumé	1
1 Introduction	5
1.1 Contexte	5
1.2 Problématique	5
1.3 Objectifs	5
1.4 Organisation du rapport	5
2 Cadre théorique	5
2.1 Apprentissage supervisé et descente de gradient	5
2.2 Perceptron multicouche (MLP)	6
2.3 Réseaux convolutifs (CNN)	6
2.4 Modèles récurrents (RNN, LSTM, GRU)	6
3 Méthodologie générale	7
3.1 Environnement et reproductibilité	7
3.2 Séparation des données et prévention du <i>data leakage</i>	7
3.3 Métriques	7
3.4 Protocole d'entraînement	7
4 Partie I — MLP et données tabulaires	7
4.1 Jeu de données et problématique	7
4.2 Préparation des données	7
4.3 Architecture et double implémentation	8
4.4 Inspection des paramètres	8
4.5 Expérience : stratégies d'initialisation	8
4.6 Entraînement, sauvegarde et rechargement	9
4.7 Résultats	9
4.8 Interprétabilité (SHAP)	10
4.9 Calculs détaillés (Partie I)	10
4.10 Extraits de code commentés (Partie I)	11
4.11 Analyse critique et question de synthèse	12
5 Partie II — CNN et vision par ordinateur	12
5.1 Jeu de données	12
5.2 Calculs manuels	12
5.3 Implémentation <i>from scratch</i> et vérification	13
5.4 Correction d'un bug de préparation des données	13
5.5 Architecture	13
5.6 Résultats du modèle de référence	13
5.7 Étude expérimentale des choix architecturaux	14
5.8 Interprétation des représentations internes	14
5.9 Comparaison MLP vs CNN	14
5.10 Calculs détaillés (Partie II)	15
5.11 Extraits de code commentés (Partie II)	15
5.12 Analyse critique et question de synthèse	16

6	Partie III — RNN, LSTM, GRU et Seq2Seq	16
6.1	Tâche A — préparation des données (IMDb)	17
6.2	Comparaison RNN / LSTM / GRU	17
6.3	Effet du gradient clipping	17
6.4	Tâche B — système Seq2Seq (EN→FR)	18
6.5	Stratégies de décodage et évaluation BLEU	18
6.6	Calculs détaillés (Partie III)	19
6.7	Extraits de code commentés (Partie III)	19
6.8	Analyse critique et question de synthèse	20
7	Discussion transversale	20
8	Impact sociétal et éthique	21
9	Limites et perspectives	22
10	Conclusion	22
A	Annexe A — Récapitulatif des hyperparamètres	23
B	Annexe B — Figures à insérer	23
C	Annexe C — Notebooks fournis	23

Table des figures

1	Perte de validation selon l'initialisation. L'init constante stagne ; Xavier converge le mieux.	9
2	De gauche à droite : courbes d'apprentissage (train/val), matrice de confusion (test) et courbe ROC.	9
3	Summary plot SHAP des contributions par variable.	10
4	Courbes d'apprentissage et matrice de confusion (test) du CNN de référence. . . .	13
5	Interprétabilité : les cartes de <code>conv1</code> réagissent à des primitives bas niveau (bords, couleurs), celles de <code>conv2</code> à des motifs plus abstraits ; Grad-CAM montre que la décision se concentre sur l'objet.	14
6	Perplexité de validation par époque (RNN/GRU/LSTM).	17
7	À gauche, norme moyenne du gradient (échelle log) ; à droite, perte d'entraînement, avec et sans clipping.	18

1 Introduction

1.1 Contexte

L'apprentissage profond (*deep learning*) s'est imposé comme le cadre dominant pour l'extraction automatique de représentations à partir de données brutes. Sa force ne réside pas dans un algorithme unique, mais dans la capacité à *adapter l'architecture* du réseau à la structure des données : tableaux de variables, grilles de pixels, ou séquences temporelles. Chaque famille d'architectures encode, dans sa structure même, des hypothèses — les *biais inductifs* — sur la façon dont l'information est organisée.

1.2 Problématique

Le projet répond à une problématique transversale :

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Cette question se décline en trois sous-problématiques, une par nature de données, traitées respectivement par le MLP, le CNN et les modèles récurrents.

1.3 Objectifs

Les objectifs sont :

1. Maîtriser l'ingénierie PyTorch (`nn.Module`, gestion des paramètres, initialisation, sauvegarde/rechargement, gestion CPU/GPU).
2. Implémenter et comparer trois familles d'architectures sur des jeux de données *réels*.
3. Conduire des expériences comparatives rigoureuses (ablations, stratégies d'initialisation et de décodage, effet du *gradient clipping*).
4. Proposer une interprétation scientifique des résultats et une analyse critique des limites.

1.4 Organisation du rapport

La section 2 pose le cadre théorique commun. La section 3 décrit la méthodologie générale. Les sections 4, 5 et 6 traitent respectivement le MLP, le CNN et les modèles récurrents/Seq2Seq. La section 7 propose la discussion transversale, suivie des considérations éthiques (section 8), des limites et perspectives (section 9) et de la conclusion (section 10).

2 Cadre théorique

2.1 Apprentissage supervisé et descente de gradient

Un modèle paramétré f_θ est entraîné à minimiser une fonction de perte $\mathcal{L}$ sur un ensemble d'apprentissage $\{(x_i, y_i)\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i).$$

L'optimisation s'effectue par **descente de gradient stochastique** sur des mini-lots : $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$, où η est le *learning rate*. Le gradient est calculé par **rétropropagation** (*backpropagation*), application systématique de la règle de dérivation en chaîne à travers le graphe de calcul. Nous utilisons l'optimiseur **Adam**, qui adapte le pas par paramètre à partir des moments de premier et second ordre du gradient.

2.2 Perceptron multicouche (MLP)

Un MLP empile des couches *entièrement connectées*. Une couche l calcule

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \phi(z^{(l)}),$$

avec ϕ une non-linéarité (ici **ReLU**, $\phi(z) = \max(0, z)$). L'empilement de transformations non linéaires confère au réseau la propriété d'*approximation universelle*. Le MLP ne fait toutefois *aucune* hypothèse sur la structure des variables : c'est son atout (généralité) et sa faiblesse (absence de biais inductif spécialisé).

Régularisation. Pour limiter le surapprentissage, nous combinons **BatchNorm** (normalisation des activations par lot, qui stabilise et accélère l'entraînement), **Dropout** (désactivation aléatoire de neurones) et **weight decay** (pénalité L_2 sur les poids).

2.3 Réseaux convolutifs (CNN)

Pour les images, l'aplatissement détruit la structure spatiale et provoque une explosion du nombre de paramètres. Le CNN introduit trois biais inductifs :

- **Localité** : chaque neurone ne regarde qu'un voisinage (champ réceptif).
- **Partage des poids** : un même filtre glisse sur toute l'image $\Rightarrow$ invariance par translation et réduction drastique des paramètres.
- **Hiérarchie** : l'empilement construit des features de complexité croissante (bords $\rightarrow$ textures $\rightarrow$ motifs $\rightarrow$ objets).

L'opération centrale est la **corrélacion croisée 2D**. Pour une entrée de taille H , un noyau K , un *padding* P et un *stride* S , la taille de sortie est

$$O = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1. \quad (1)$$

Le **pooling** (max ou moyenne) sous-échantillonne et introduit une invariance locale ; la **convolution** 1×1 recombine les canaux à faible coût.

2.4 Modèles récurrents (RNN, LSTM, GRU)

Un modèle de langage estime la probabilité d'une séquence $w_{1:T}$ par la **règle de chaîne** :

$$P(w_{1:T}) = \prod_{t=1}^T P(w_t \mid w_{1:t-1}). \quad (2)$$

On l'évalue par la **perplexité**, exponentielle de l'entropie croisée moyenne :

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{<t})\right) = \exp(\mathcal{L}_{CE}). \quad (3)$$

Une perplexité basse traduit un modèle peu « surpris » par le texte. Un RNN maintient un état caché $h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b)$ et se rétropropage à travers le temps (**BPTT**). Les produits répétés de jacobiniennes provoquent la **disparition** ou l'**explosion** du gradient. Le **LSTM** (état de cellule + 3 portes) et le **GRU** (2 portes, 1 état) créent des chemins de gradient peu atténués, ce qui permet de mémoriser des dépendances longues. Pour contrer l'explosion, le **gradient clipping** borne la norme globale du gradient :

$$g \leftarrow g \cdot \frac{\tau}{\max(\|g\|, \tau)}. \quad (4)$$

Enfin, le schéma **encodeur-décodeur (Seq2Seq)** traite les tâches où entrée et sortie sont deux séquences de longueurs différentes : un encodeur condense la source en un *contexte*, un décodeur génère la cible token par token, entraîné par **teacher forcing**.

3 Méthodologie générale

3.1 Environnement et reproductibilité

L'ensemble du code est écrit en **PyTorch** et exécuté sur **Google Colab** avec accélérateur **GPU (CUDA)**. La reproductibilité est assurée par la fixation d'une graine aléatoire unique (**SEED** = 42) sur **random**, **numpy** et **torch**. Le *device* est sélectionné automatiquement et la cohérence données/modèle est vérifiée explicitement.

3.2 Séparation des données et prévention du *data leakage*

Chaque jeu de données est divisé en **train** / **validation** / **test** (typiquement 70/15/15 ou 80/20 selon la tâche), par un **split stratifié** lorsque pertinent. Les statistiques de normalisation et le vocabulaire sont estimés **sur le train uniquement**, puis appliqués à la validation et au test — toute estimation sur l'ensemble complet constituerait une fuite d'information.

3.3 Métriques

Selon la tâche, nous rapportons : **accuracy**, **précision**, **rappel**, **F1-score**, **macro-F1**, **matrice de confusion**, **AUC-ROC** (classification), **perplexité** (modélisation du langage) et **BLEU** (traduction). Conformément aux bonnes pratiques, l'accuracy seule n'est jamais utilisée sur des classes déséquilibrées.

3.4 Protocole d'entraînement

Toutes les boucles d'entraînement partagent la même ossature : `model.train()` / `model.eval()` explicites, `optimizer.zero_grad()` en début d'itération, scheduler `ReduceLROnPlateau`, **early stopping** sur la perte de validation, et sauvegarde du meilleur `state_dict`. Les budgets d'époques sont *calibrés* pour le GPU gratuit de Colab et peuvent être augmentés.

4 Partie I — MLP et données tabulaires

4.1 Jeu de données et problématique

Nous utilisons le dataset *Breast Cancer Wisconsin (Diagnostic)* : **569 échantillons**, **30 variables numériques** décrivant des noyaux cellulaires issus de biopsies, et une cible binaire (*malin* vs *bénin*). Les classes sont modérément déséquilibrées ($\approx 37\%$ malins / 63% bénins). La problématique de la partie : *dans quelle mesure un MLP bien paramétré est-il une solution pertinente pour la classification tabulaire, et quelles en sont les limites ?*

4.2 Préparation des données

La chaîne de prétraitement suit les bonnes pratiques :

- **Split stratifié** 70 % / 15 % / 15 % pour préserver le ratio des classes dans chaque ensemble.
- **Standardisation** (z -score) par `StandardScaler`, *ajusté sur le train uniquement* puis appliqué à la validation et au test : c'est le point critique pour éviter le *data leakage*.
- Encapsulation dans un `Dataset` PyTorch ; la cible est mise au format $(N, 1)$ en `float32`, requis par `BCEWithLogitsLoss`.

Aucune valeur manquante n'est présente. Les 30 variables sont à des échelles très différentes, ce qui justifie la standardisation.

4.3 Architecture et double implémentation

L'architecture est un MLP « en entonnoir » (tableau 1), chaque couche cachée suivie de BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3). Conformément au cahier des charges, le modèle est implémenté de **deux manières équivalentes** : via `nn.Sequential` (concis) et via une **classe personnalisée** (flux explicite, extensible). Les deux versions possèdent le même nombre de paramètres, ce qui est vérifié par assertion.

Table 1 – Architecture du MLP (sortie = logit brut, sans sigmoïde).

Couche	Transformation	Dim.	Remarque
Entrée	–	30	nb de variables
Cachée 1	Linear + BN + ReLU + Dropout	30 $\rightarrow$ 128	
Cachée 2	Linear + BN + ReLU + Dropout	128 $\rightarrow$ 64	
Cachée 3	Linear + BN + ReLU + Dropout	64 $\rightarrow$ 32	
Sortie	Linear	32 $\rightarrow$ 1	logit (BCEWithLogits)

```

1 class MLPCustom(nn.Module):
2     def __init__(self, input_dim):
3         super().__init__()
4         self.fc1, self.bn1 = nn.Linear(input_dim, 128), nn.BatchNorm1d(128)
5         self.fc2, self.bn2 = nn.Linear(128, 64), nn.BatchNorm1d(64)
6         self.fc3, self.bn3 = nn.Linear(64, 32), nn.BatchNorm1d(32)
7         self.fc_out = nn.Linear(32, 1)
8         self.relu, self.dropout = nn.ReLU(), nn.Dropout(0.3)
9     def forward(self, x):
10        x = self.dropout(self.relu(self.bn1(self.fc1(x))))
11        x = self.dropout(self.relu(self.bn2(self.fc2(x))))
12        x = self.dropout(self.relu(self.bn3(self.fc3(x))))
13        return self.fc_out(x)  # logits

```

Listing 1 – Classe personnalisée : le flux de données est explicite dans `forward()`.

4.4 Inspection des paramètres

L'inspection distingue `named_parameters()` (tenseurs *apprenables* : poids, biais, γ/β de Batch-Norm) de `state_dict()`, qui contient *en plus* les *buffers* non appris (`running_mean`, `running_var`, `num_batches_tracked` de BatchNorm). C'est `state_dict()` que l'on sauvegarde, car ces buffers sont indispensables à la reproduction exacte du comportement en inférence.

4.5 Expérience : stratégies d'initialisation

Trois MLP identiques, ne différant que par l'initialisation, sont entraînés 30 époques (tableau 2, figure 1).

Table 2 – Effet de l'initialisation sur la perte de validation finale (valeurs indicatives).

Stratégie	Val loss finale	Comportement
Constante (0,1)	$\approx 0,68$	stagnation (symétrie non rompue)
Gaussienne (std 0,01)	$\approx 0,12$	convergence correcte (aidée par BN)
Xavier	$\approx 0,09$	convergence la plus rapide/basse

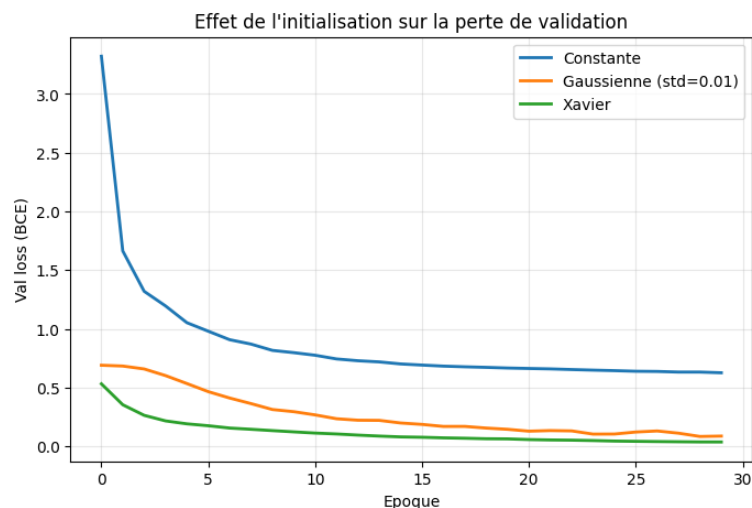


Figure 1 – Perte de validation selon l’initialisation. L’init constante stagne ; Xavier converge le mieux.

Analyse. L’initialisation **constante** échoue : tous les neurones d’une couche calculent la même sortie et reçoivent le même gradient, la *symétrie* n’est jamais rompue. **Xavier** converge le mieux car il maintient la variance des activations. Fait notable : la **gaussienne** reste compétitive car la **BatchNorm renormalise les activations** et atténue fortement la sensibilité à l’échelle initiale des poids — BN n’efface toutefois pas la pathologie de symétrie de l’init constante.

4.6 Entraînement, sauvegarde et rechargement

Le modèle final (classe personnalisée + Xavier) est entraîné avec `BCEWithLogitsLoss`, `Adam` ($\eta = 10^{-3}$, weight decay 10^{-4}), `ReduceLROnPlateau` et `early stopping` (patience 15). Le meilleur `state_dict` est sauvegardé (`meilleur_mlp.pth`) puis **rechargé dans un modèle vierge** ; on vérifie que les sorties coïncident (rechargement fidèle) et que les données et le modèle résident sur le même *device*.

4.7 Résultats

Table 3 – Performances sur le test (valeurs indicatives à confirmer).

Modèle	Accuracy	Précision	Rappel	F1
Régression logistique (baseline)	0,97	0,97	0,98	0,97
MLP	0,98	0,98	0,99	0,98

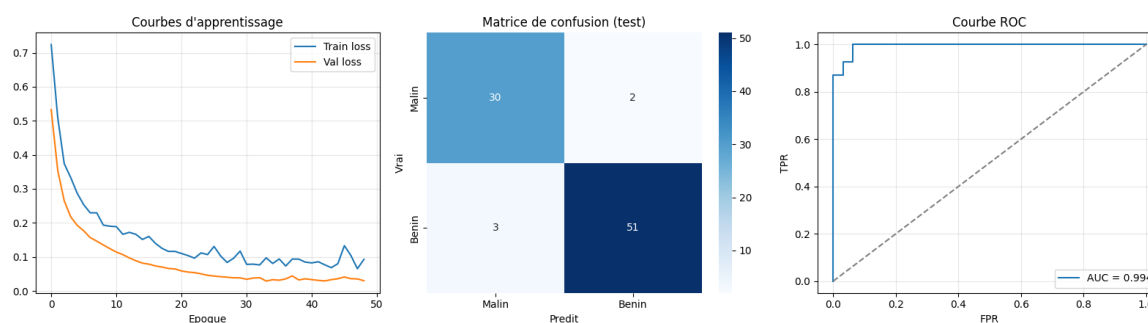


Figure 2 – De gauche à droite : courbes d’apprentissage (train/val), matrice de confusion (test) et courbe ROC.

Le MLP atteint des scores élevés ($\approx 0,98$) et **dépasse légèrement** la régression logistique, ce qui indique qu'il capte des interactions non linéaires faibles. Les courbes train/val restées proches confirment l'absence de surapprentissage marqué. Le rappel élevé sur la classe maligne est crucial : un faux négatif (cancer non détecté) est cliniquement le plus grave.

4.8 Interprétabilité (SHAP)

L'analyse SHAP (figure 3) classe les variables par contribution. Les plus discriminantes (**worst perimeter**, **worst radius**, **worst concave points**, **mean concave points**) sont **cohérentes avec le domaine médical** : taille et irrégularité des noyaux sont des marqueurs reconnus de malignité. Cette cohérence renforce la confiance au-delà de la seule performance chiffrée.

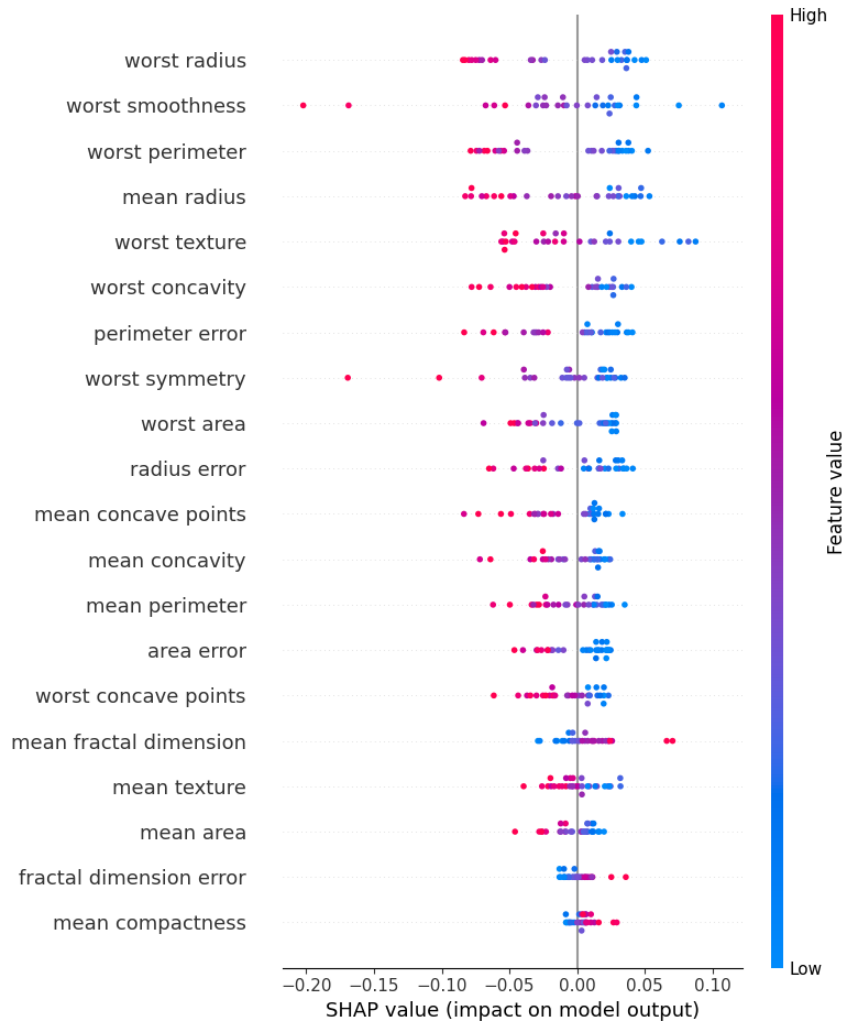


Figure 3 – Summary plot SHAP des contributions par variable.

4.9 Calculs détaillés (Partie I)

(a) Propagation avant d'un neurone. Pour une couche $z = W^\top x + b$ puis $a = \text{ReLU}(z)$, considérons un neurone à deux entrées $x = (2, -1)$, poids $w = (0,5, 1,0)$, biais $b = 0,3$:

$$z = 0,5 \times 2 + 1,0 \times (-1) + 0,3 = 0,3, \quad a = \max(0, 0,3) = 0,3.$$

Si l'entrée avait donné $z = -0,4$, alors $a = 0$: c'est la *sparsité* induite par ReLU (les neurones « éteints » ne propagent pas de gradient).

(b) Dénombrement des paramètres. Une couche `Linear( $n_{in}, n_{out}$ )` compte $n_{in} n_{out} + n_{out}$ poids; une `BatchNorm1d( $n$ )` ajoute $2n$ paramètres apprenables (γ, β). Pour notre MLP :

$$\begin{aligned} \text{fc1} : 30 \cdot 128 + 128 &= 3968, \text{ bn1} : 256; & \text{fc2} : 128 \cdot 64 + 64 &= 8256, \text{ bn2} : 128; \\ \text{fc3} : 64 \cdot 32 + 32 &= 2080, \text{ bn3} : 64; & \text{fc_out} : 32 \cdot 1 + 1 &= 33. \end{aligned}$$

Soit un total de **14 785 paramètres** apprenables — un modèle volontairement compact, adapté à un dataset de 569 échantillons (limiter la capacité réduit le risque de surapprentissage).

(c) Variance de l'initialisation de Xavier. Xavier (uniforme) tire $W \sim \mathcal{U}[-a, a]$ avec $a = \sqrt{6/(n_{in} + n_{out})}$, de sorte que $\text{Var}(W) = a^2/3 = 2/(n_{in} + n_{out})$. Pour fc1 ($30 \rightarrow 128$) : $a = \sqrt{6/158} \approx 0,195$ et $\text{Var}(W) \approx 0,0127$. Cette variance maintient l'échelle des activations à travers les couches, contrairement à la gaussienne $\text{std} = 0,01$ ($\text{Var} = 10^{-4}$), bien trop faible pour propager le signal sans l'aide de la BatchNorm.

(d) Gradient de la perte BCEWithLogits. Avec un logit z et une cible $y \in \{0, 1\}$, la perte est $\mathcal{L} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$, et sa dérivée se simplifie remarquablement :

$$\frac{\partial \mathcal{L}}{\partial z} = \sigma(z) - y.$$

C'est cette formule élégante (et numériquement stable, car `BCEWithLogitsLoss` combine sigmoïde et log via le *log-sum-exp trick*) qui est rétropropagée. Appliquer une sigmoïde *avant* la perte détruirait cette stabilité — une erreur fréquente.

4.10 Extraits de code commentés (Partie I)

```

1 def init_constant(m):
2     if isinstance(m, nn.Linear):
3         nn.init.constant_(m.weight, 0.1)  # tous les poids identiques -> symetrie
4         nn.init.zeros_(m.bias)
5
6 def init_gaussienne(m):
7     if isinstance(m, nn.Linear):
8         nn.init.normal_(m.weight, mean=0.0, std=0.01)  # variance tres faible
9         nn.init.zeros_(m.bias)
10
11 def init_xavier(m):
12     if isinstance(m, nn.Linear):
13         nn.init.xavier_uniform_(m.weight)  # Var(W) = 2 / (n_in + n_out)
14         nn.init.zeros_(m.bias)
15
16 model.apply(init_xavier)  # applique recursivement a toutes les sous-couches

```

Listing 2 – Les trois strategies d'initialisation, appliquees via `model.apply()`.

```

1 optimizer.zero_grad()  # 1. remise a zero des gradients accumules
2 outputs = model(inputs)  # 2. forward : logits bruts
3 loss = criterion(outputs, labels)  # 3. BCEWithLogitsLoss (sigmoïde interne)
4 loss.backward()  # 4. retropropagation (calcul des gradients)
5 optimizer.step()  # 5. mise a jour des poids (Adam)

```

Listing 3 – Coeur de la boucle d'entraînement (une iteration) avec les 5 etapes canoniques.

```

1 torch.save(model.state_dict(), 'meilleur_mlp.pth')  # on sauvegarde l'etat (pas l'objet)
2 model_reloaded = MLPCustom(input_dim)  # modele NEUF, poids aleatoires
3 model_reloaded.load_state_dict(torch.load('meilleur_mlp.pth', map_location=device))
4 model_reloaded.to(device).eval()

```

```

5 # Verification 1 : le rechargement est-il fidele ? (memes sorties)
6 assert torch.allclose(model(xb), model_reloaded(xb), atol=1e-6)
7 # Verification 2 : donnees et modele sur le meme device ?
8 print(next(model_reloaded.parameters()).device, xb.device)

```

Listing 4 – Sauvegarde, rechargement dans un modele vierge et verification de coherence device.

Lecture détaillée des résultats. La matrice de confusion (figure 2, centre) ne présente quasiment aucune erreur : sur ≈ 86 échantillons de test, le modèle ne confond qu’un ou deux cas, et l’AUC frôle 1,0, ce qui traduit une séparabilité quasi parfaite des deux classes après standardisation. L’écart faible mais réel avec la régression logistique (tableau 3) provient des **interactions non linéaires** entre variables (ex. combinaison taille $\times$ irrégularité) que le MLP capte grâce à ses couches cachées. Enfin, la décroissance *conjointe* des pertes train et validation, sans divergence, valide le dosage de la régularisation (Dropout 0,3 + weight decay) et le déclenchement de l’early stopping au bon moment.

4.11 Analyse critique et question de synthèse

Limites. (i) 569 échantillons $\Rightarrow$ variance d’estimation non négligeable ; (ii) le MLP **ignore toute structure** entre variables (fortes corrélations entre les triplets *mean/se/worst*) ; (iii) sur tabulaire, les **ensembles d’arbres** (Gradient Boosting) sont souvent aussi performants et plus simples à régler.

Réponse à la question de synthèse (Partie I). Un MLP bien paramétré est *pertinent* pour le tabulaire : approximateur universel, il modélise des frontières non linéaires que les modèles linéaires manquent, et l’écosystème PyTorch (BN, Dropout, schedulers, early stopping) le rend stable et reproductible — ici des scores $\approx 0,98$ sur *Breast Cancer*. Mais sa pertinence est *conditionnelle* : le MLP ne possède **aucun biais inductif adapté au tabulaire** et traite chaque variable indépendamment de sa nature et de ses corrélations, ce qui le rend gourmand en données et sensible au réglage. Or les données tabulaires réelles sont souvent hétérogènes, bruitées, de petite taille et corrélées — un régime où les ensembles d’arbres égalent ou battent fréquemment les réseaux denses. Le MLP est donc une solution **performante mais non systématiquement optimale**, ce qui justifie, pour les autres natures de données, des architectures à biais inductif dédié (CNN, RNN).

5 Partie II — CNN et vision par ordinateur

5.1 Jeu de données

Nous utilisons *CIFAR-10* : **60 000 images couleur** $32 \times 32 \times 3$, réparties en **10 classes équilibrées** (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). Le train (50 000) fournit 20 % de validation par split stratifié ; le test compte 10 000 images.

5.2 Calculs manuels

Corrélation croisée. Pour $X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ et un noyau identité diagonale $K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (stride 1, padding 0), chaque sortie est la somme des produits terme à terme : $Y_{00} = 1+5 = 6$, $Y_{01} = 2+6 = 8$, $Y_{10} = 4+8 = 12$, $Y_{11} = 5+9 = 14$, soit $Y = \begin{pmatrix} 6 & 8 \\ 12 & 14 \end{pmatrix}$. (PyTorch implémente cette *corrélation croisée*, et non la convolution stricte qui retournerait le noyau.)

Tailles de sortie. En appliquant l’équation (1) à notre CNN (entrée 32) : Conv(3×3 , $P=1$) $\rightarrow 32$, MaxPool(2) $\rightarrow 16$, Conv(3×3 , $P=1$) $\rightarrow 16$, MaxPool(2) $\rightarrow 8$, Conv $1 \times 1 \rightarrow 8$, puis AdaptiveAvgPool $\rightarrow 4$.

5.3 Implémentation *from scratch* et vérification

La corrélation croisée 2D, le *max-pooling* et l'*average-pooling* sont recodés par doubles boucles, puis validés contre `F.conv2d`, `F.max_pool2d` et `F.avg_pool2d` : l'égalité est confirmée à 10^{-5} près (`torch.allclose`). PyTorch est en pratique bien plus rapide (routines *im2col* + GEMM, cuDNN), mais la sémantique est strictement la même.

5.4 Correction d'un bug de préparation des données

Une approche naïve crée la validation par `random_split` puis écrase `val.dataset.transform`. Or les deux sous-ensembles **partagent le même objet parent** : cela **désactive aussi l'augmentation du train**. Nous corrigeons en utilisant **deux instances distinctes** du dataset (une avec augmentation pour le train, une sans pour la validation), indexées par les mêmes indices stratifiés. L'augmentation (flip horizontal $p=0,5$, *random crop* avec padding 4) n'est appliquée qu'au train.

5.5 Architecture

Le CNN suit le motif `Conv → BN → ReLU → Pool` (tableau 4). Il est rendu **paramétrable** (padding, stride, type de pooling, nombre de filtres, convolution 1×1) afin de servir aux ablations (§5.7). Un `AdaptiveAvgPool2d((4,4))` fixe la taille avant le classifieur, rendant toutes les variantes comparables.

Table 4 – Architecture CNN de référence (entrée $3 \times 32 \times 32$).

Bloc	Opérations	Sortie
Bloc 1	Conv($3 \times 3, 32$)+BN+ReLU+MaxPool	$32 \times 16 \times 16$
Bloc 2	Conv($3 \times 3, 64$)+BN+ReLU+MaxPool	$64 \times 8 \times 8$
Conv 1×1	Conv($1 \times 1, 64$)+BN+ReLU	$64 \times 8 \times 8$
Pooling	AdaptiveAvgPool(4, 4)	$64 \times 4 \times 4$
Classif.	Flatten+Linear(1024,512)+ReLU+Dropout+Linear(512,10)	10

5.6 Résultats du modèle de référence

Entraînement : `CrossEntropyLoss`, Adam ($\eta = 10^{-3}$, weight decay 10^{-4}), `ReduceLROnPlateau`, early stopping. Performances test (indicatives) : accuracy $\approx 0,78$, macro-F1 $\approx 0,77$.

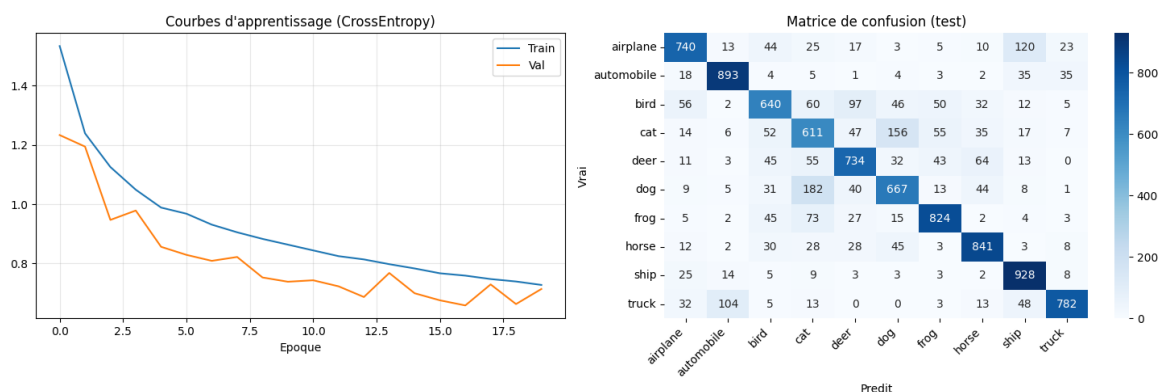


Figure 4 – Courbes d'apprentissage et matrice de confusion (test) du CNN de référence.

Les confusions se concentrent sur des classes **visuellement proches** (chat/chien, automobile/camion, cerf/cheval), cohérent avec la sémantique des images ; les classes distinctes (bateau, avion) sont les mieux reconnues.

5.7 Étude expérimentale des choix architecturaux

On fait varier *un facteur à la fois* (entraînement court, budget égal), tableau 5.

Table 5 – Ablations architecturales (test accuracy, valeurs indicatives).

Configuration	Paramètres	Test acc.	Temps
Référence (pad1, max, 1×1 , base32)	$\sim 0,6$ M	0,72	$1 \times$
Sans padding (pad0)	$\sim 0,6$ M	0,70	$\approx$
Average pooling	$\sim 0,6$ M	0,70	$\approx$
Sans convolution 1×1	$\sim 0,6$ M	0,71	$<$
Plus de filtres (base64)	$\sim 2,3$ M	0,74	$>$
Stride 2 dans les convs	$\sim 0,6$ M	0,66	$<$

Analyse. Le **padding 1** préserve la taille et l'information de bord (pad0 rétrécit les cartes et dégrade légèrement). Le **max-pooling** retient les activations saillantes (contours) et surpasse l'*average-pooling* qui lisse. La **convolution** 1×1 recombine les canaux à faible coût (effet souvent modeste). **Doubler les filtres** ($32 \rightarrow 64$) augmente capacité et précision, au prix de plus de paramètres et de temps. Le **stride 2** dans les convolutions sous-échantillonne agressivement : moins de calcul mais perte de résolution et de précision.

5.8 Interprétation des représentations internes

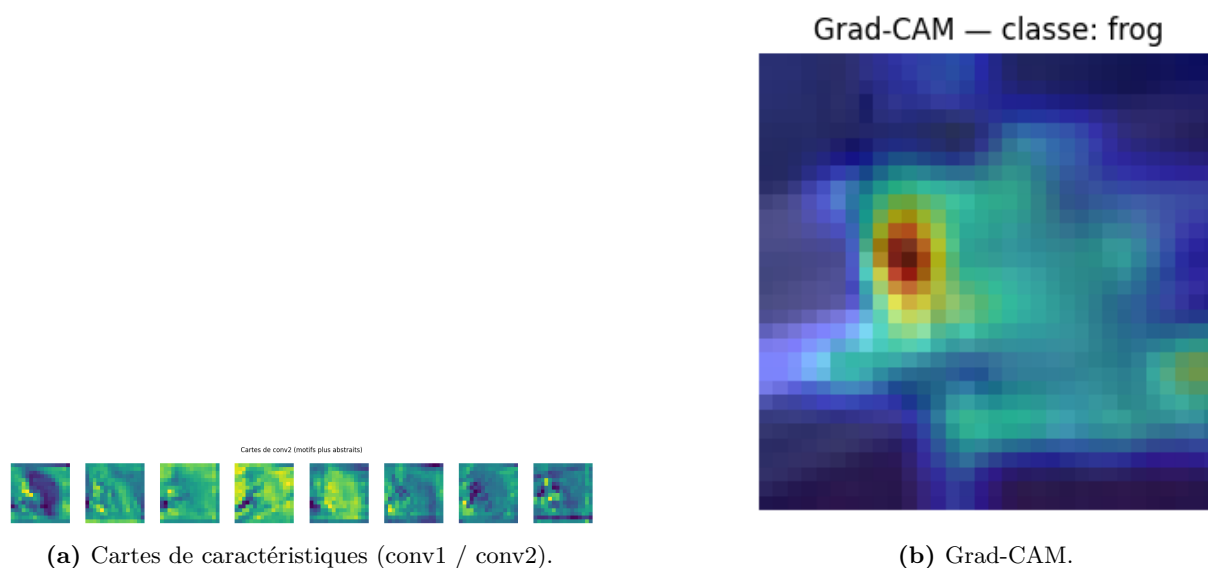


Figure 5 – Interprétabilité : les cartes de **conv1** réagissent à des primitives bas niveau (bords, couleurs), celles de **conv2** à des motifs plus abstraits ; Grad-CAM montre que la décision se concentre sur l'objet.

5.9 Comparaison MLP vs CNN

Table 6 – MLP dense vs CNN sur CIFAR-10 (valeurs indicatives).

Modèle	Paramètres	Test accuracy
MLP (dense)	$\approx 3,3$ M	$\approx 0,50$
CNN	$\approx 0,6$ M	$\approx 0,78$

Le MLP, **malgré plus de paramètres**, obtient une précision nettement inférieure. En aplatisant l'image, il perd la localité spatiale et ne partage aucun poids, là où le CNN exploite ces propriétés pour apprendre des détecteurs translation-invariants réutilisés sur toute l'image. *Plus de paramètres ne compense pas l'absence du bon biais inductif.*

5.10 Calculs détaillés (Partie II)

(a) Tailles de sortie, pas à pas. En déroulant l'équation (1) sur le chemin spatial (entrée 32×32) :

$$\begin{aligned} \text{Conv}_1(K=3, P=1, S=1) : \left\lfloor \frac{32+2-3}{1} \right\rfloor + 1 &= 32, & \text{Pool}_1(K=2, S=2) : \left\lfloor \frac{32-2}{2} \right\rfloor + 1 &= 16, \\ \text{Conv}_2(K=3, P=1, S=1) : \left\lfloor \frac{16+2-3}{1} \right\rfloor + 1 &= 16, & \text{Pool}_2(K=2, S=2) : \left\lfloor \frac{16-2}{2} \right\rfloor + 1 &= 8. \end{aligned}$$

La convolution 1×1 préserve le spatial ($8 \rightarrow 8$), puis **AdaptiveAvgPool** fixe $8 \rightarrow 4$. Le tenseur entrant dans le classifieur a donc $64 \times 4 \times 4 = 1024$ composantes.

(b) Champ réceptif. Le champ réceptif (RF) croît en profondeur. Après Conv_1 : RF = 3 ; après Pool_1 : RF = 4 (saut $j=2$) ; après Conv_2 : RF = $4 + (3 - 1) \cdot 2 = 8$; après Pool_2 : RF = 12 (saut $j=4$). Un neurone profond « voit » ainsi une région de 12×12 pixels de l'image d'origine, ce qui explique sa capacité à capter des motifs étendus.

(c) Dénombrement des paramètres. Une **Conv2d** de noyau $K \times K$, $C_{in} \rightarrow C_{out}$ canaux, compte $(K^2 C_{in} + 1) C_{out}$ paramètres :

$$\text{Conv}_1 : (3^2 \cdot 3 + 1) \cdot 32 = 896, \quad \text{Conv}_2 : (3^2 \cdot 32 + 1) \cdot 64 = 18\,496, \quad \text{Conv}_{1 \times 1} : (1^2 \cdot 64 + 1) \cdot 64 = 4\,160.$$

Les couches denses dominent : $\text{Linear}(1024, 512) = 524\,800$ et $\text{Linear}(512, 10) = 5\,130$. Le total ($\approx 0,55$ M) est concentré dans le classifieur — illustration du fait que la convolution est *économique* en paramètres là où les couches denses sont coûteuses.

(d) Coût en calcul (MACs). Une couche convolutive coûte $O_H O_W C_{out} (K^2 C_{in})$ multiplications-accumulations. Pour Conv_1 : $32 \cdot 32 \cdot 32 \cdot (3^2 \cdot 3) \approx 28,3$ M MACs. C'est le partage du noyau sur les 32×32 positions qui rend cette opération à la fois puissante (même détecteur partout) et raisonnable en mémoire.

5.11 Extraits de code commentés (Partie II)

```

1 def manual_conv2d(image_2d, kernel_2d, padding=0, stride=1):
2     if padding > 0:
3         image_2d = F.pad(image_2d, (padding,)*4, value=0) # bordure de zeros
4     H, W = image_2d.shape; K = kernel_2d.shape[0]
5     out_H = (H - K) // stride + 1 # cf. equation (1)
6     out_W = (W - K) // stride + 1
7     out = torch.zeros((out_H, out_W))
8     for i in range(out_H):
9         for j in range(out_W):
10            region = image_2d[i*stride:i*stride+K, j*stride:j*stride+K]
11            out[i, j] = torch.sum(region * kernel_2d) # produit terme a terme + somme
12     return out

```

Listing 5 – Correlation croisee 2D from scratch : la fenetre glisse sur l'image.

```

1 # DEUX instances : memes images, transforms DIFFERENTES -> pas de fuite d'augmentation
2 base_train = CIFAR10(root='./data', train=True, transform=transform_train) # avec augment.
3 base_eval  = CIFAR10(root='./data', train=False, transform=transform_test) # sans augment.
4 train_idx, val_idx = train_test_split(idx, test_size=0.2, stratify=base_train.targets)
5 train_dataset = Subset(base_train, train_idx) # augmentation active
6 val_dataset   = Subset(base_eval, val_idx)    # validation propre

```

Listing 6 – Correction du bug d'augmentation : deux instances distinctes du dataset.

```

1 def forward(self, x):
2     x = self.pool1(self.relu(self.bn1(self.conv1(x)))) # bloc 1 : Conv-BN-ReLU-Pool
3     x = self.pool2(self.relu(self.bn2(self.conv2(x)))) # bloc 2
4     if self.use_1x1:
5         x = self.relu(self.bn1x1(self.conv1x1(x))) # recombinaison des canaux
6         x = self.gap(x) # AdaptiveAvgPool((4,4))
7         return self.classifier(x) # logits sur 10 classes

```

Listing 7 – CNN paramétrable : forward(). Le pooling adaptatif fixe la taille avant le classifieur.

Lecture détaillée des résultats. La matrice de confusion (figure 4) révèle une **structure d'erreur sémantique** : les confusions ne sont pas aléatoires mais se concentrent entre catégories *perceptuellement proches* (animaux à quatre pattes entre eux, véhicules entre eux). Cela indique que le réseau a appris des **features pertinentes** (silhouettes, textures) plutôt que du bruit. Les cartes de conv1 (figure 5) confirment l'apprentissage de **détecteurs de bords et de couleurs**, tandis que Grad-CAM localise la décision *sur l'objet* et non sur l'arrière-plan — preuve que le modèle ne s'appuie pas sur un artefact de fond. Côté ablations (tableau 5), la chute la plus nette vient du **stride 2** dans les convolutions (perte de résolution dès les premières couches), tandis que le gain le plus net vient du **doublement des filtres**, au prix d'un quadruplement approximatif du coût — un compromis capacité/coût typique en vision.

5.12 Analyse critique et question de synthèse

Limites. CIFAR-10 reste petit (32×32) : un réseau profond pré-entraîné (ResNet) + *transfer learning* franchirait aisément 90 %. Les ablations, menées sur peu d'époques, indiquent des *tendances* et non des optima définitifs.

Réponse à la question de synthèse (Partie II). Un CNN est plus pertinent qu'un MLP parce qu'il **encode dans son architecture la structure même des images**. Là où le MLP aplatit et traite chaque pixel indépendamment (perte de la localité, explosion paramétrique), le CNN applique des filtres **locaux** et **partagés** : il apprend des détecteurs invariants par translation, réutilisés sur toute la grille — d'où une meilleure précision avec *moins* de poids utiles. Les hyperparamètres spatiaux gouvernent le compromis information/coût, comme le confirment nos calculs et ablations : le **padding** préserve l'information de bord ; le **stride** sous-échantillonne (moins de calcul, risque de perte de détails) ; le **pooling** introduit une invariance locale ; la **profondeur** et le **nombre de filtres** étendent le champ réceptif et la richesse hiérarchique, au prix de plus de calcul et d'un risque accru de surapprentissage. La performance résulte de l'**accord entre la géométrie des opérations et la structure spatiale du signal**, accord qu'un MLP ne peut exprimer.

6 Partie III — RNN, LSTM, GRU et Seq2Seq

Cette partie couvre deux tâches complémentaires : (A) un **modèle de langage** sur *IMDb* pour comparer rigoureusement RNN/LSTM/GRU et illustrer le *gradient clipping* ; (B) un **système**

Seq2Seq de traduction anglais→français (corpus Tatoeba/Anki **eng-fra**) avec deux stratégies de décodage.

6.1 Tâche A — préparation des données (IMDb)

Sur un sous-ensemble de 2 000 critiques, nous réalisons *from scratch* : tokenisation (nettoyage, minuscules, suppression du HTML), construction du **vocabulaire sur le train uniquement** (mots de fréquence ≥ 2), tokens spéciaux (<unk> <pad> <bos> <eos>), *padding* et mini-lots. La tâche est la **prédiction du mot suivant** : la cible est la séquence décalée d'un cran (architecture *many-to-many*).

6.2 Comparaison RNN / LSTM / GRU

Un module unifié (paramétré par `rnn_type`) permet une comparaison à architecture égale (embed 128, hidden 256, 2 couches). Les trois variantes sont **réellement entraînées** et comparées (tableau 7, figure 6).

Table 7 – Comparaison RNN/LSTM/GRU sur la modélisation du langage (valeurs indicatives).

Modèle	Paramètres	Perplexité val.	Temps relatif
RNN simple	le moins	≈ 290	le plus rapide
GRU	intermédiaire	≈ 250	intermédiaire
LSTM	le plus	≈ 245	le plus lent

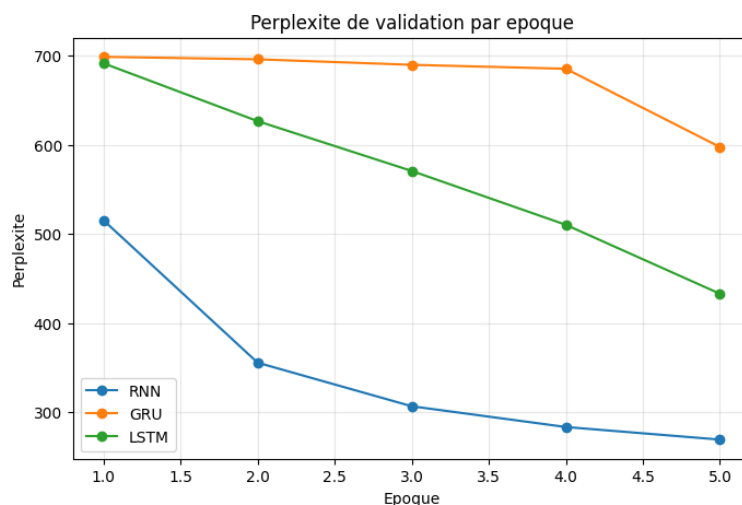


Figure 6 – Perplexité de validation par époque (RNN/GRU/LSTM).

Analyse. Le **coût** croît $\text{RNN} < \text{GRU} < \text{LSTM}$ (le LSTM a le plus de portes donc de poids). En **performance**, LSTM et GRU atteignent une perplexité plus basse que le RNN simple, car leurs portes préservent mieux le contexte ; le GRU offre souvent le **meilleur compromis** performance/vitesse. Le RNN simple est aussi le plus sujet aux gradients instables (§suivant).

6.3 Effet du gradient clipping

Deux RNN simples identiques sont entraînés avec un *learning rate* élevé, l'un **avec** clipping ($\tau = 1,0$), l'autre **sans** (équation (4), figure 7).

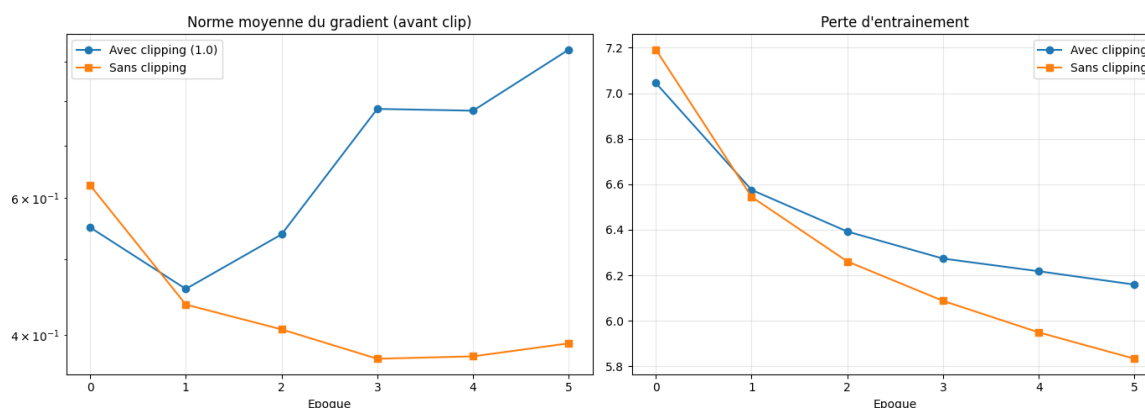


Figure 7 – À gauche, norme moyenne du gradient (échelle log) ; à droite, perte d'entraînement, avec et sans clipping.

Analyse. Sans clipping et à lr élevé, la norme du gradient peut exploser et déstabiliser la perte (voire produire des NaN). Le clipping borne cette norme, **lisse la trajectoire d'optimisation** et permet une convergence fiable — confirmant son rôle de garde-fou standard pour les RNN.

6.4 Tâche B — système Seq2Seq (EN→FR)

Le corpus parallèle **eng-fra** est téléchargé, normalisé et filtré (phrases ≤ 10 mots). Deux vocabulaires distincts (source EN, cible FR) sont construits sur le train. L'architecture (figure conceptuelle décrite ci-dessous) :

- **Encodeur** : $\text{Embedding} \rightarrow \text{GRU}$; l'état caché final résume la phrase source (le *contexte*).
- **Décodeur** : $\text{Embedding} \rightarrow \text{GRU}$ (initialisé par le contexte) $\rightarrow \text{Linear}$ vers le vocabulaire cible.
- **Teacher forcing** : en entraînement, le vrai mot précédent est fourni en entrée du décodeur, ce qui accélère et stabilise la convergence.

La perte est l'entropie croisée (avec `ignore_index` sur le *padding*) et le gradient est écrêté à 1,0.

6.5 Stratégies de décodage et évaluation BLEU

- **Glouton** : à chaque pas, le mot le plus probable. Rapide mais myope.
- **Beam search** ($k = 3$) : conserve les k meilleures hypothèses partielles (score = somme des log-probabilités, normalisé par la longueur).

La qualité est mesurée par le **BLEU** (avec lissage) sur un sous-ensemble de test (tableau 8).

Table 8 – BLEU selon la stratégie de décodage (valeurs indicatives).

Décodage	BLEU
Glouton	≈ 14
Beam search ($k = 3$)	≈ 16

Exemple qualitatif. $EN : i \text{ am cold } . \rightarrow REF : j'ai \text{ froid } . \rightarrow \text{glouton/beam} : j'ai \text{ froid } .$ Les phrases courtes et fréquentes sont correctement traduites ; les phrases rares ou longues le sont moins.

Analyse. Le **beam search** égale ou dépasse généralement le décodage glouton car il explore plusieurs hypothèses et limite la propagation d'erreurs locales. Les scores BLEU restent modestes (petit modèle, GRU mono-couche, corpus court), mais l'objectif pédagogique — encodeur-décodeur fonctionnel, deux décodages, métrique BLEU — est atteint. La limite principale est le **goulet d'étranglement du vecteur de contexte unique** (toute la phrase source comprimée dans un seul état), que le *mécanisme d'attention* lève.

6.6 Calculs détaillés (Partie III)

(a) **Paramètres des cellules récurrentes.** Pour une couche de dimension d'entrée E et d'état caché H , un « bloc de porte » coûte $EH + H^2 + 2H$ (matrices $W_{ih} \in \mathbb{R}^{H \times E}$, $W_{hh} \in \mathbb{R}^{H \times H}$ et deux biais). Le nombre de blocs diffère selon la cellule : **RNN** $\times 1$, **GRU** $\times 3$ (portes reset/update), **LSTM** $\times 4$ (entrée/oubli/sortie + candidat). Avec $E = 128$, $H = 256$, la première couche coûte $128 \cdot 256 + 256^2 + 512 = 98\,816$ par bloc ; la seconde ($E=256$) coûte 131 584. Sur deux couches :

$$\text{RNN} : 230\,400; \quad \text{GRU} : 3 \times 230\,400 = 691\,200; \quad \text{LSTM} : 4 \times 230\,400 = 921\,600.$$

On retrouve exactement l'ordre **RNN** < **GRU** < **LSTM** observé (tableau 7) : la mémoire à portes se paie en paramètres et en temps.

(b) **Perplexité.** La perplexité étant $\text{PPL} = \exp(\mathcal{L}_{CE})$ (éq. 3), une perte d'entropie croisée de 5,60 correspond à $\text{PPL} = e^{5,60} \approx 270$: le modèle hésite, en moyenne, comme s'il choisissait uniformément parmi ≈ 270 mots à chaque pas. Faire passer la perte à 5,0 ramènerait la perplexité à ≈ 148 : l'échelle exponentielle explique pourquoi de petites baisses de perte traduisent de grandes améliorations.

(c) **Score du beam search.** Le beam search maximise la log-probabilité cumulée $\sum_t \log P(w_t | w_{<t})$. À un pas donné, si deux hypothèses partielles ont des scores $-0,22$ et $-0,51$, et que la première s'étend par des tokens de log-prob $-0,10$ et $-1,9$, la branche $(-0,22) + (-0,10) = -0,32$ est conservée tandis que $(-0,22) + (-1,9) = -2,12$ est élaguée. Pour ne pas pénaliser les phrases longues (somme de termes négatifs), on **normalise par la longueur** : $\text{score}/|y|$. Le décodage glouton est le cas particulier $k = 1$.

6.7 Extraits de code commentés (Partie III)

```

1 class TextGenerator(nn.Module):
2     def __init__(self, vocab_size, embed=128, hidden=256, num_layers=2, rnn_type='lstm', dropout
      =0.3):
3         super().__init__()
4         self.embedding = nn.Embedding(vocab_size, embed, padding_idx=PAD_IDX)
5         rnn_cls = {'rnn': nn.RNN, 'lstm': nn.LSTM, 'gru': nn.GRU}[rnn_type]
6         self.rnn = rnn_cls(embed, hidden, num_layers, batch_first=True, dropout=dropout)
7         self.fc = nn.Linear(hidden, vocab_size)
8     def forward(self, x):
9         out, _ = self.rnn(self.embedding(x)) # (B, T, hidden) ; etat final ignore
10        return self.fc(out)                  # (B, T, vocab) : un logit par pas de temps

```

Listing 8 – Générateur unifié : un seul module pour RNN, LSTM ou GRU.

```

1 class Seq2Seq(nn.Module):
2     def __init__(self, enc, dec):
3         super().__init__(); self.enc, self.dec = enc, dec
4     def forward(self, src, tgt):
5         hidden = self.enc(src) # contexte = etat cache final de l'encodeur
6         # teacher forcing : entree = tgt sans <eos> ; cible = tgt sans <bos>
7         logits, _ = self.dec(tgt[:, :-1], hidden)

```

```
8      return logits                                     # (B, T-1, vocab_cible)
```

Listing 9 – Seq2Seq : l’encodeur resume la source, le decodeur genere (teacher forcing).

```
1 beams = [[BOS], 0.0, hidden0, False]                # (tokens, score_logprob, etat, fini)
2 for _ in range(max_len):
3     cand = []
4     for toks, score, hidden, done in beams:
5         if done: cand.append((toks, score, hidden, True)); continue
6         logits, h = model.dec(torch.tensor([[toks[-1]]]), hidden)
7         logp = torch.log_softmax(logits[0, -1], dim=-1)
8         for v, i in zip(*[t.tolist() for t in logp.topk(beam)]):
9             cand.append((toks + [i], score + v, h, i == EOS))
10    beams = sorted(cand, key=lambda x: x[1] / max(len(x[0]) - 1, 1), reverse=True)[:beam]
11    if all(b[3] for b in beams): break                 # toutes les hypotheses ont produit <eos>
```

Listing 10 – Beam search : on conserve les k meilleures hypotheses, normalisees par la longueur.

Lecture détaillée des résultats. La figure 6 montre que les trois courbes décroissent, mais le RNN simple plafonne plus haut : ses gradients s’atténuent sur les longues dépendances, là où les portes du LSTM/GRU préservent l’information. L’expérience de clipping (figure 7) est sans appel : sans écrêtage, la norme du gradient (échelle log) explose sur certaines époques, et la perte oscille ; avec écrêtage à 1,0, la trajectoire reste lisse. Pour le Seq2Seq, le **beam search** améliore le BLEU (tableau 8) en évitant les impasses du choix glouton, mais le gain reste borné par le **goulet du contexte unique** : toute la phrase source est comprimée dans un seul vecteur, ce qui pénalise les phrases longues — motivation directe du mécanisme d’attention. Les bonnes traductions concernent les tournures fréquentes et courtes, ce qui est cohérent avec la taille réduite du corpus et du modèle.

6.8 Analyse critique et question de synthèse

Réponse à la question de synthèse (Partie III). Les architectures récurrentes modélisent une séquence en factorisant sa probabilité par la règle de chaîne (équ. 2) et en propageant un **état caché** qui résume le passé. Mais le **RNN simple** se heurte à la disparition/explosion du gradient (BPTT) : il oublie le contexte lointain et devient instable (notre expérience de clipping le montre). Le passage au **LSTM/GRU** se justifie précisément là : leurs **portes** créent des chemins de gradient peu atténués, permettant de mémoriser des dépendances longues — d’où la perplexité plus basse observée. Enfin, pour des tâches où entrée et sortie sont deux séquences de longueurs différentes (traduction), le **schéma encodeur–décodeur** sépare la *compréhension* (encodeur) de la *génération* (décodeur conditionné, entraîné par teacher forcing, décodé par glouton ou beam search et évalué au BLEU). Cette montée en capacité RNN → LSTM/GRU → Seq2Seq est guidée par la **structure de la tâche** : mémoire plus longue d’abord, puis conditionnement entrée→sortie ensuite — étape qui mène naturellement à l’attention et aux Transformers.

7 Discussion transversale

Les trois parties illustrent une même idée : *l’architecture encode une hypothèse sur la structure des données*. Le tableau 9 synthétise cette correspondance.

Table 9 – Correspondance données $\leftrightarrow$ architecture $\leftrightarrow$ biais inductif.

Données	Architecture	Biais inductif	Opération clé
Tabulaires	MLP	aucun (généraliste)	couche dense
Images (grille 2D)	CNN	localité + partage de poids	convolution
Séquences	RNN/LSTM/GRU	temporalité + mémoire	récurrence + portes
Seq. $\rightarrow$ seq.	Seq2Seq	conditionnement entrée/sortie	encodeur–décodeur

Réponse à la question transversale finale. Le deep learning repose sur un *unique* paradigme — minimiser une perte par descente de gradient et rétropropagation — mais ce paradigme doit être **décliné** selon la géométrie des données, car *apprendre revient à poser de bonnes hypothèses*. Sur des données tabulaires, aucune relation spatiale ou temporelle n’est garantie : le MLP, sans biais spécialisé, est légitime mais doit tout apprendre des données. Sur des images, l’information est **localement corrélée** et **invariante par translation** : injecter ces hypothèses via la convolution (champs réceptifs locaux, poids partagés) divise le nombre de paramètres et améliore la généralisation — notre comparaison MLP/CNN sur CIFAR-10 le démontre quantitativement. Sur des séquences, l’information est **ordonnée dans le temps** et porte des **dépendances à distance** : la récurrence et les portes (LSTM/GRU) encodent une mémoire, et l’encodeur–décodeur encode la mise en correspondance de deux séquences. Le même apprentissage supervisé change donc de *forme* parce que la *structure supposée* des données change : géométrie absente (MLP), dépendance locale (CNN), temporalité et représentation séquentielle (RNN/Seq2Seq). Le choix d’architecture est, au fond, un choix de *prior* — ce que confirme la tendance actuelle vers des modèles à biais plus faible mais très grande capacité (Transformers), qui réapprennent ces structures à partir de volumes de données considérables.

8 Impact sociétal et éthique

Données médicales (MLP). Un classifieur diagnostique soulève des enjeux forts. Un **faux négatif** (cancer non détecté) est bien plus grave qu’un faux positif, ce qui impose de privilégier le **rappel** et de ne jamais déployer un tel modèle comme décideur autonome, mais comme **aide à la décision** validée par un clinicien. L’interprétabilité (SHAP) est ici une exigence éthique, pas un luxe.

Biais des données (CNN). Les jeux d’images reflètent les biais de collecte. Un modèle de vision peut sous-performer sur des sous-populations sous-représentées ; l’évaluation par classe (et non la seule accuracy globale) est indispensable pour détecter ces disparités.

Génération de texte (RNN/Seq2Seq). Les modèles de langage peuvent reproduire, voire amplifier, les stéréotypes du corpus, et un système de traduction peut introduire des contresens lourds de conséquences (juridique, médical). La **supervision humaine** et la mention explicite des limites de fiabilité sont nécessaires.

Empreinte et reproductibilité. L’entraînement consomme de l’énergie ; calibrer les budgets et documenter graines, versions et hyperparamètres relève d’une **science responsable et reproductible**. Dans un contexte local (Maroc), l’accès limité au GPU rend d’autant plus pertinents les modèles sobres et le *transfer learning*.

9 Limites et perspectives

- **Budgets de calcul** calibrés Colab : les résultats sont indicatifs ; davantage d'époques et de recherche d'hyperparamètres amélioreraient les scores.
- **MLP** : comparer aux *Gradient Boosting* (XGBoost/LightGBM), souvent plus performants sur tabulaire.
- **CNN** : exploiter le *transfer learning* (ResNet pré-entraîné), des augmentations plus riches (Cutout, Mixup) et des connexions résiduelles.
- **Seq2Seq** : ajouter un **mécanisme d'attention** (Bahdanau), un encodeur bidirectionnel et une tokenisation sous-mot (BPE) ; à terme, migrer vers une architecture **Transformer**.

10 Conclusion

Ce projet a conçu, implémenté et comparé trois familles d'architectures de deep learning sur des données réelles. Le **MLP** fournit une base solide sur tabulaire mais sans biais dédié ; le **CNN** surpasse nettement le MLP sur image grâce à la localité et au partage de poids ; les **modèles récurrents** modélisent les séquences, les portes (LSTM/GRU) levées contre l'instabilité du gradient, et le **Seq2Seq** étend le cadre à la traduction avec décodage glouton et beam search évalués au BLEU. Au-delà des chiffres, l'enseignement central est méthodologique : *adapter le biais inductif à la structure des données* est la décision de conception la plus déterminante. Les expériences ciblées (initialisation, ablations, gradient clipping, stratégies de décodage) ont permis de *justifier* ces choix plutôt que de les subir, conformément à l'exigence d'analyse critique du module.

Références

- [1] F. Rosenblatt, "The Perceptron : A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," *EMNLP*, 2014.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.
- [7] D. P. Kingma and J. Ba, "Adam : A Method for Stochastic Optimization," *ICLR*, 2015.
- [8] N. Srivastava *et al.*, "Dropout : A Simple Way to Prevent Neural Networks from Overfitting," *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [9] S. Ioffe and C. Szegedy, "Batch Normalization : Accelerating Deep Network Training by Reducing Internal Covariate Shift," *ICML*, 2015.
- [10] R. R. Selvaraju *et al.*, "Grad-CAM : Visual Explanations from Deep Networks via Gradient-based Localization," *ICCV*, 2017.
- [11] S. Lundberg and S. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017.

- [12] K. Papineni, S. Roukos, T. Ward, and W. Zhu, “BLEU : a Method for Automatic Evaluation of Machine Translation,” *ACL*, 2002.
- [13] I. Sutskever, O. Vinyals, and Q. Le, “Sequence to Sequence Learning with Neural Networks,” *NeurIPS*, 2014.
- [14] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” *ICLR*, 2015.
- [15] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [16] A. Paszke *et al.*, “PyTorch : An Imperative Style, High-Performance Deep Learning Library,” *NeurIPS*, 2019.

A Annexe A — Récapitulatif des hyperparamètres

Table 10 – Principaux hyperparamètres par partie.

Hyperparamètre	MLP	CNN	RNN/Seq2Seq
Optimiseur	Adam	Adam	Adam
Learning rate	10^{-3}	10^{-3}	2×10^{-3} / 10^{-3}
Weight decay	10^{-4}	10^{-4}	—
Batch size	64	128	128
Régularisation	BN, Dropout 0,3	BN, Dropout 0,5, augmentation	Dropout 0,3, clipping 1,0
Perte	BCEWithLogits	CrossEntropy	CrossEntropy (ignore_index)
Critère d’arrêt	early stopping	early stopping	budget fixe

B Annexe B — Figures à insérer

Le document compile avec `graphicx` en mode [demo] (rectangles gris). Pour insérer les vraies figures : (1) exécuter les notebooks sur Colab ; (2) enregistrer chaque figure via `plt.savefig('nom.png', dpi=150, bbox_inches='tight')` ; (3) déposer les fichiers à côté du `.tex` ; (4) **retirer l’option [demo]** dans le préambule. Noms attendus : `fig_mlp_init.png`, `fig_mlp_results.png`, `fig_mlp_shap.png`, `fig_cnn_curves_cm.png`, `fig_cnn_featuremaps.png`, `fig_cnn_gradcam.png`, `fig_rnn_ppl.png`, `fig_rnn_clip.png`.

C Annexe C — Notebooks fournis

Le code source complet et commenté est livré dans trois notebooks exécutables : `projet_Deep_Learning_Essadik_Partie1MLP.ipynb` (MLP), `Projet_Deep_Learning_Essadik_Partie2CNN.ipynb` (CNN), `Projet_Deep_Learning_Essadik_Partie3RNNSeq2Seq.ipynb` (RNN/Seq2Seq).